Development of a PX4-Based Drone Simulator with LiDAR Obstacle Avoidance Using ROS2 and Gazebo

Monu Kumar
Department of Computer Science & Artificial Intelligence
Rishihood University, India
monu.k23csai@nst.rishihood.edu.in
Enrollment No: 230113

***Abstract*—**This paper describes the design and implementation of a fully integrated drone simulation environment that combines the PX4 autopilot, Gazebo (Ignition) physics engine, and ROS2 Humble for autonomous obstacle avoidance. A quadrotor model equipped with a 2D LiDAR sensor (x500_lidar_2d) is simulated in a realistic world. A custom ROS2 node subscribes to the LiDAR scan, implements a keyboard teleoperation interface, and provides automatic collision avoidance when obstacles are detected within 1.5 m. The system overcomes common simulation challenges such as heading estimation errors, failsafe triggers, and offboard control mode configuration. The final solution uses the microRTPS agent to bridge ROS2 velocity setpoints to PX4's uORB bus, enabling responsive offboard control. The complete setup is reproducible and serves as a foundation for advanced autonomous navigation research.

***Keywords*—**PX4 autopilot; Gazebo simulation; ROS2; LiDAR; obstacle avoidance; UAV; SITL; offboard control; microRTPS.

I. INTRODUCTION

Unmanned aerial vehicles (UAVs) are increasingly used in research, agriculture, and surveillance. Before real-world deployment, simulation is essential for algorithm validation and safety testing. The PX4 open-source autopilot, together with the Gazebo simulator, provides a powerful Software-In-The-Loop (SITL) environment. However, integrating ROS2 for sensor processing and offboard control requires careful configuration of multiple components.

This paper presents a step-by-step solution that resolves common pitfalls (failsafe activation, invalid heading estimates, lost setpoints) and yields a fully functional drone simulator with real-time obstacle avoidance. The remainder of this paper is organized as follows: Section II describes the system architecture; Section III details the hardware and software stack; Section IV presents the implementation; Section V discusses results; and Section VI concludes the paper.

II. SYSTEM ARCHITECTURE

The system comprises four interacting layers that work in concert to deliver a seamless simulation experience:

Simulation Layer — Gazebo (Ignition) renders the drone, LiDAR sensor, and environment with full physics simulation.

Flight Control Layer — PX4 SITL runs the autopilot and estimator (EKF2), managing attitude,

IV. IMPLEMENTATION DETAILS

A. PX4 Parameter Configuration

To enable offboard velocity control and suppress simulation-specific failsafes, the following parameters were set permanently via the PX4 shell. These parameters eliminate the "heading estimate invalid" and "battery warning" errors that otherwise prevent arming and offboard mode entry:

```
param set COM_LOW_BAT_ACT 0
param set COM_DISARM_PRFLT 0
param set NAV_RCL_ACT 0
param set CBRK_SUPPLY_CHK 894281
param set COM_OF_LOSS_T 10
param set COM_ARM_WO_GPS 1
param set EKF2_MAG_TYPE 5
param set COM_ARM_MAG_ANG -1
param set SYS_HAS_MAG 0
param save
```

B. microRTPS Bridge Construction

The px4_ros_com package was built from source to obtain the micrortps_agent binary. The workspace was established using the colcon build system and launched with UDP transport to minimize latency:

```
mkdir -p ~/ws_offboard/src
cd ~/ws_offboard/src
git clone
https://github.com/PX4/px4_msgs.git
git clone
```

velocity, and position control loops.

Bridge Layer — microRTPS agent forwards ROS2 messages to PX4's uORB, and ros_gz_bridge relays LiDAR scans from Gazebo to ROS2 topics.

Application Layer — Custom ROS2 nodes handle obstacle detection, velocity control, and keyboard teleoperation via Python.

The component interaction follows a publish-subscribe model. Gazebo publishes LiDAR scans which are relayed by ros_gz_bridge to a ROS2 topic. The obstacle avoidance node consumes this topic, computes velocity commands, and publishes them. The microRTPS agent bridges these commands to PX4's internal uORB message bus for execution.

III. HARDWARE & SOFTWARE STACK

All experiments were conducted on Ubuntu 22.04 LTS. Table I lists the complete software stack with version information.

TABLE I. Software Stack and Version Information

Component	Version / Source
PX4 Autopilot	main branch (April 2026 commit)
Gazebo (Ignition)	gz-sim8 (Fortress / Harmonic)
ROS2	Humble Hawksbill
ROS2–PX4 Bridge	px4_msgs + px4_ros_com (source build)
Gazebo–ROS2 Bridge	ros_gz_bridge (ROS2 official)
Drone Model	x500_lidar_2d (2D LiDAR)
Custom Nodes	Python 3 (rclpy)
Build System	colcon, cmake

```
https://github.com/PX4/px4_ros_com.git
cd ~/ws_offboard && colcon build
micrortps_agent -t UDP
```

C. Obstacle Avoidance Node

The custom node (obstacle_avoidance_node.py) subscribes to the LiDAR scan topic and divides each 360° scan into four sectors: front, back, left, and right. The minimum range in each sector is compared against a safety threshold of 1.5 m. When a threat is detected, an avoidance velocity vector is published to override the pilot's input. The node also manages an automatic takeoff sequence upon mode activation.

D. Offboard Mode Fix Node

A parallel node (offboard_fix) continuously publishes the correct OffboardControlMode message at 10 Hz on /fmu/in/offboard_control_mode. This ensures that PX4 never times out waiting for offboard setpoints, which would otherwise trigger a failsafe and return-to-home action.

E. Launch Procedure

The reproducible workflow requires five parallel terminals. Table II summarizes the command for each terminal. After arming and entering offboard mode, the operator presses R to initiate takeoff and WASD for translational control.

V. RESULTS

The implemented system successfully achieves stable offboard velocity control with immediate response to keyboard inputs. Automatic takeoff produces a configurable vertical climb of 10 m. Obstacle detection and avoidance reliably triggers when any LiDAR sector detects a return closer than 1.5 m, causing the drone to reverse or translate sideways. After applying the PX4 parameters in Section IV-A, no failsafe interruptions were observed during extended simulation runs.

VI. CONCLUSION

This paper presented a complete and reproducible drone simulation stack integrating PX4, Gazebo (Ignition), and ROS2 Humble. The primary contributions are: (1) a systematic method to resolve common SITL issues including heading estimation failure, battery failsafe, and offboard setpoint loss; (2) a lightweight Python-based obstacle avoidance node with keyboard override capability; and (3) a clear five-terminal workflow that serves as a baseline for further research.

Future work will extend the system with a stereo camera and deep-learning-based collision prediction, as well as integration of a global path planner to enable fully

autonomous waypoint navigation in cluttered environments.

TABLE II. Five-Terminal Launch Procedure

Terminal	Command
1	`make px4_sitl gz_x500_lidar_2d → commander arm -f → commander mode offboard`
2	`ros2 run ros_gz_bridge parameter_bridge "/world/.../scan@sensor_msgs/msg/LaserScan@gz.msgs.LaserScan"`
3	`micrortps_agent -t UDP (source workspace first)`
4	`ros2 run offboard_fix fix_offboard`
5	`ros2 run obstacle_avoidance obstacle_avoidance_node`

REFERENCES

[1] L. Meier, D. Honegger, and M. Pollefeys, “PX4: A node-based autonomous flight control system,” 2015.

[2] Open Source Robotics Foundation, “Gazebo: Robot simulation made easy,” [Online]. Available: https://gazebosim.org.

[3] ROS2 Development Team, “Robot Operating System 2 (ROS2) Documentation,” [Online]. Available: https://docs.ros.org.

[4] PX4 Development Team, “PX4 User Guide – Gazebo Simulation,” [Online]. Available: https://docs.px4.io.

[5] M. Quigley et al., “ROS: an open-source Robot Operating System,” ICRA Workshop on Open Source Software, 2009.

[6] A. S. Huang et al., “Open source for mobile and aerial robotics,” IEEE Robotics & Automation Magazine, 2020.